Airflow REST API Workflow - Complete Technical Documentation

Executive Summary

This document provides a comprehensive line-by-line breakdown of an Apache Airflow DAG that orchestrates 3 sequential REST API calls with Kafka integration. The workflow validates input configurations, normalizes parameters, executes HTTP requests, and publishes telemetry events to Kafka for monitoring and tracking.

Key Features:

- JSON schema-based input validation
- Global and task-specific configuration merging
- Sequential REST API execution with error handling
- XCom-based data passing between tasks
- Kafka event publishing (start/success/failure)
- Resilient finalization (runs even if API calls fail)

Table of Contents

1. Architecture Overview
2. Data Flow Diagram
3. Configuration Structure
4. Detailed Code Walkthrough
5. REST API Call Mechanism
6. XCom Data Passing
7. Kafka Integration
8. Error Handling Strategy
9. Example Execution Scenarios

Architecture Overview

Workflow Stages

The DAG follows a linear pipeline with 6 main stages:

1. **Kafka Start Event** - Publish workflow initiation event
2. **Input Preparation** - Validate and normalize configuration
3. **REST API Execution** - Sequential execution of 3 HTTP calls
4. **Result Finalization** - Aggregate results and identify failures
5. **Kafka End Event** - Publish success/failure event with summary

Task Dependency Chain

publish_kafka_start
↓
prepare_inputs
↓
task1_rest_api_call
↓
task2_rest_api_call
↓
task3_rest_api_call
↓
finalize_results
↓
publish_final_status

Critical Design Pattern: The finalization and final Kafka tasks use `TriggerRule.ALL_DONE`, ensuring they execute regardless of upstream failures. This guarantees that completion events are always published.

Data Flow Diagram

Input Configuration Sources

Configuration Loading Priority

1. **dag_run.conf** (Primary Source)
    - Passed when triggering DAG via UI, CLI, or API
    - Contains JSON payload with correlation_id and Task1-3 configs
2. **context['params']** (Fallback Source)
    - Used if dag_run.conf is empty or absent
    - Typically defined in DAG definition

XCom Data Flow Between Tasks

Source Task	XCom Key	Consumer Tasks
prepare_inputs	(default)	task1/2/3_rest_api_call
task1_rest_api_call	(default)	finalize_results
task2_rest_api_call	(default)	finalize_results
task3_rest_api_call	(default)	finalize_results
finalize_results	(default)	publish_final_status

Table 1: XCom data passing relationships

XCom Mechanics:[1]

- **Push:** When a Python function returns a value, Airflow automatically pushes it to XCom
- **Pull:** Downstream tasks retrieve values using `ti.xcom_pull(task_ids='task_name')`
- **Storage:** XCom data is stored in Airflow's metadata database (PostgreSQL/MySQL)
- **Scope:** XCom values are scoped to a specific DAG run instance

---

Configuration Structure

Expected Input Schema

The DAG expects a JSON configuration with this structure:

```
{
"correlation_id": "unique-request-id-12345",
"global": {
"headers": {
"Authorization": "Bearer token123",
"User-Agent": "Airflow/2.0"
},
"timeout": 45,
"verify_ssl": true
},
"Task1": {
"url": "https://api.example.com/users",
"method": "GET",
"headers": {
"Accept": "application/json"
},
"timeout": 30
},
"Task2": {
"url": "https://api.example.com/orders",
"method": "POST",
"json": {
"user_id": 123,
"product": "widget"
}
},
"Task3": {
"url": "https://api.example.com/notifications",
"method": "PUT",
"json": {
"message": "Order processed"
},
"verify_ssl": false
}
}
```

Schema Validation Rules

Required Fields (Top Level):

- `correlation_id` - Unique identifier for request tracing (minimum 1 character)
- `Task1`, `Task2`, `Task3` - Configuration objects for each API call

Required Fields (Each Task):

- `url` - HTTP endpoint (must be non-empty string starting with http:// or https://)
- `method` - HTTP verb (GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS)

Optional Fields (Global):

- `headers` - Dictionary of HTTP headers applied to all tasks
- `timeout` - Request timeout in seconds (default: 30)
- `verify_ssl` - SSL certificate verification (default: true)

Optional Fields (Each Task):

- `headers` - Task-specific headers (merged with global, overrides global)
- `json` - Request body payload (can be object, array, string, number, boolean, null)
- `timeout` - Task-specific timeout (overrides global)
- `verify_ssl` - Task-specific SSL setting (overrides global)

Detailed Code Walkthrough

Section 1: Imports and Constants

Import Block Analysis

```
import json
import logging
from datetime import datetime, timedelta, timezone
from typing import Any, Dict, List, Optional
```

Purpose: Standard Python libraries for JSON handling, logging, datetime operations, and type hints.

```
import jsonschema
from jsonschema import ValidationError
```

Purpose: Validates input configuration against the defined JSON schema. Ensures all required fields are present and correctly typed before workflow execution.

```
from airflow import DAG
from airflow.exceptions import AirflowException
from airflow.sdk.definitions.context import get_current_context
from airflow.models.dagrun import DagRun
from airflow.providers.apache.kafka.operators.produce import
ProduceToTopicOperator
from airflow.providers.standard.operators.python import
PythonOperator
from airflow.utils.trigger_rule import TriggerRule
```

Purpose: Core Airflow components:

- `DAG` - Main workflow container
- `AirflowException` - Standard exception for workflow failures
- `get_current_context()` - Retrieves Airflow context (dag_run, ti, run_id) within task functions
- `DagRun` - Type hint for DAG run object
- `ProduceToTopicOperator` - Kafka message publishing operator
- `PythonOperator` - Executes Python functions as tasks
- `TriggerRule` - Controls when tasks execute based on upstream states

Constant Definitions

```
default_args = {
'owner': 'airflow',
'depends_on_past': False,
'email_on_failure': False,
'email_on_retry': False,
'retries': 1,
'retry_delay': timedelta(minutes=5),
```

```
'trigger_rule': TriggerRule.ALL_SUCCESS,
}
```

Explanation:

- `owner` - Task ownership for UI display
- `depends_on_past` - If True, task only runs if previous run succeeded
- `email_on_failure/retry` - Email notification settings
- `retries` - Number of automatic retry attempts (1 = one retry after initial failure)
- `retry_delay` - Wait time between retries
- `trigger_rule` - Default rule: task runs only if all upstream tasks succeeded

```
KAFKA_EXECUTION_TIMEOUT = timedelta(seconds=120)
KAFKA_POLL_TIMEOUT = 60.0
```

Kafka Timeouts:

- `KAFKA_EXECUTION_TIMEOUT` - Maximum time for Kafka producer to complete
- `KAFKA_POLL_TIMEOUT` - Time to wait for Kafka broker acknowledgment

```
EVENT_TYPE_RUN_STARTED = "run.started.v1"
EVENT_TYPE_RUN_SUCCEEDED = "run.succeeded.v1"
EVENT_TYPE_RUN_FAILED = "run.failed.v1"
EVENT_STATUS_RUNNING = "RUNNING"
EVENT_STATUS_SUCCEEDED = "SUCCEEDED"
EVENT_STATUS_FAILED = "FAILED"
OUTCOME_SUCCESS = "success"
OUTCOME_FAILED = "failed"
```

Event Taxonomy: Standardized event types and statuses for workflow telemetry. The versioned event types (`v1`) enable schema evolution over time.

```
DEFAULT_TIMEOUT_SECONDS = 30
DEFAULT_VERIFY_SSL = True
VALID_HTTP_METHODS = {'GET', 'POST', 'PUT', 'PATCH', 'DELETE',
'HEAD', 'OPTIONS'}
```

HTTP Configuration:

- DEFAULT_TIMEOUT_SECONDS - Applied when not specified in global or task config
- DEFAULT_VERIFY_SSL - Enforces SSL certificate validation by default
- VALID_HTTP_METHODS - Whitelist of supported HTTP verbs

Schema Definition

```
_TASK_PROPERTIES = {
'url': {'type': 'string', 'minLength': 1},
'method': {'type': 'string'},
'headers': {'type': 'object'},
'json': {'type': ['object', 'array', 'string', 'number', 'boolean', 'null']},
'timeout': {'type': 'number', 'minimum': 1},
'verify_ssl': {'type': 'boolean'},
}
```

Property Definitions: JSON schema validation rules for individual task properties. Note that `json` accepts multiple types (object, array, primitives, null) to support diverse API payloads.

```
CONFIG_SCHEMA = {
'type': 'object',
'properties': {
'correlation_id': {'type': 'string', 'minLength': 1},
'global': {
'type': 'object',
'properties': {
'headers': {'type': 'object'},
'timeout': {'type': 'number', 'minimum': 1},
'verify_ssl': {'type': 'boolean'},
},
'additionalProperties': True,
},
**{f'Task{n}': {
'type': 'object',
'required': ['url', 'method'],
'properties': _TASK_PROPERTIES,
'additionalProperties': True,
```

```
} for n in range(1, 4)},
},
'required': ['correlation_id', 'Task1', 'Task2', 'Task3'],
'additionalProperties': True,
}
```

Schema Construction: Uses dictionary unpacking (`**`) to generate Task1, Task2, Task3 definitions programmatically. Each task requires `url` and `method`, while other properties are optional. The `additionalProperties: True` allows extra fields (ignored by validator but available to custom logic).

Section 2: Utility Functions

Header Merging Function

```
def _merge_headers(
correlation_id: str,
global_headers: Optional[Dict[str, str]] = None,
task_headers: Optional[Dict[str, str]] = None,
) -> Dict[str, str]:
base = {'X-Correlation-Id': correlation_id, 'Content-Type':
'application/json'}
if isinstance(global_headers, dict):
base.update(global_headers)
if isinstance(task_headers, dict):
base.update(task_headers)
return base
```

Merge Strategy:

1. **Base Headers** - Always include X-Correlation-Id (for distributed tracing) and Content-Type
2. **Global Headers** - Applied second, can override base headers
3. **Task Headers** - Applied last, can override global and base headers

Example Merge:

Input:

```
correlation_id = "req-123"
global_headers = {"Authorization": "Bearer token", "User-Agent": "Airflow"}
task_headers = {"User-Agent": "Custom-Client"}
```

Output:

```
{
"X-Correlation-Id": "req-123",
"Content-Type": "application/json",
"Authorization": "Bearer token",
"User-Agent": "Custom-Client" # Task overrides global
}
```

HTTP Method Validation

```
def _validate_http_method(method: str) -> None:
m = (method or '').upper()
if m not in VALID_HTTP_METHODS:
raise ValueError(f"Invalid HTTP method '{method}'. Supported: ({', '.join(sorted(VALID_HTTP_METHODS))})")
```

Validation Logic:

1. Convert to uppercase (accepts lowercase input like "get" or "post")
2. Check against whitelist
3. Raise descriptive error with supported methods if invalid

Why Uppercase? HTTP methods are case-sensitive in the spec (RFC 7231), but converting to uppercase normalizes input and matches Python requests library convention.

URL Validation

```
def _validate_url(url: str) -> None:
if not isinstance(url, str) or not url.strip():
raise ValueError("URL must be a non-empty string")
u = url.lower().strip()
if not (u.startswith('http://') or u.startswith('https://')):
raise ValueError("URL must start with http:// or https://")
```

Two-Stage Validation:

1. Type and emptiness check (rejects None, empty strings, non-strings)
2. Protocol check (only allows HTTP/HTTPS, rejects FTP, file://, etc.)

Security Consideration: This prevents Server-Side Request Forgery (SSRF) attacks using file:// or internal protocols.

Global Defaults Extraction

```
def _extract_global_defaults(config: Dict[str, Any]) -> Dict[str, Any]:
g = config.get('global') or {}
timeout = g.get('timeout', DEFAULT_TIMEOUT_SECONDS)
if not isinstance(timeout, (int, float)) or timeout < 1:
timeout = DEFAULT_TIMEOUT_SECONDS
verify_ssl = g.get('verify_ssl', DEFAULT_VERIFY_SSL)
if not isinstance(verify_ssl, bool):
verify_ssl = DEFAULT_VERIFY_SSL
headers = g.get('headers') if isinstance(g.get('headers'), dict) else {}
return {'headers': headers or {}, 'timeout': float(timeout), 'verify_ssl':
verify_ssl}
```

Safe Default Extraction:

- Uses `.get()` with fallbacks to avoid KeyError exceptions
- Type validation with fallback to defaults if invalid (e.g., timeout="invalid" → 30)
- Converts timeout to float for consistency with requests library
- Ensures headers is always a dictionary (even if not provided)

Defense Against Bad Input: This function is defensive - it won't crash if global config is malformed or missing, instead falling back to safe

defaults.

Task Configuration Normalization

def _normalize_task_config(
task_config: Dict[str, Any],
task_key: str,
global_defaults: Dict[str, Any],
correlation_id: str,
) -> Dict[str, Any]:
if not isinstance(task_config, dict):
raise AirflowException(f"{task_key} configuration must be a dictionary")
if 'url' not in task_config:
raise AirflowException(f"{task_key} missing required field: url")
if 'method' not in task_config:
raise AirflowException(f"{task_key} missing required field: method")
_validate_url(task_config['url'])
_validate_http_method(task_config['method'])

```
timeout = task_config.get('timeout', global_defaults['timeout'])
if not isinstance(timeout, (int, float)) or timeout < 1:
    timeout = global_defaults['timeout']

verify_ssl = task_config.get('verify_ssl', global_defaults['verify_ssl'])
if not isinstance(verify_ssl, bool):
    verify_ssl = global_defaults['verify_ssl']

headers = _merge_headers(
    correlation_id,
    global_defaults['headers'],
    task_config.get('headers'),
)

return {
    'url': task_config['url'].strip(),
    'method': task_config['method'].upper(),
    'headers': headers,
    'json': task_config.get('json'),
```

```
    'timeout': float(timeout),
    'verify_ssl': verify_ssl,
}
```

Normalization Process:

1. **Required Field Check** - Validates url and method presence
2. **URL/Method Validation** - Calls validation functions
3. **Override Logic** - Task config overrides global defaults
4. **Type Safety** - Falls back to global if task value invalid
5. **Header Merge** - Combines correlation_id + global + task headers
6. **Output Format** - Returns standardized dictionary ready for requests library

Output Example:

Normalized task config ready for HTTP execution:

{
'url': 'https://api.example.com/endpoint',
'method': 'POST',
'headers': {
'X-Correlation-Id': 'req-123',
'Content-Type': 'application/json',
'Authorization': 'Bearer token'
},
'json': {'key': 'value'},
'timeout': 30.0,
'verify_ssl': True
}

Section 3: Workflow Functions

Configuration Validation

def validate_request_payload(**context) -> Dict[str, Any]:
dag_run = context.get('dag_run')
request_payload = dag_run.conf if dag_run and dag_run.conf else {}
if not request_payload:
request_payload = context.get('params', {})

```
logger.debug("Received request payload: %s", json.dumps(request_payload, i

if not isinstance(request_payload, dict):
    raise AirflowException("Configuration must be a dictionary")
if not request_payload:
    raise AirflowException("Configuration is empty. Provide correlation_id an

try:
    jsonschema.validate(instance=request_payload, schema=CONFIG_SCHEMA
except ValidationError as e:
    msg = str(e)
    if e.path:
        msg += f" (at: {' -> '.join(str(p) for p in e.path)})"
    logger.error("Configuration validation failed: %s", msg)
    raise AirflowException(f"Configuration validation failed: {msg}") from e

logger.info("Request payload validation successful")
return request_payload
```

Configuration Loading Priority:[2]

1. **dag_run.conf** - Primary source when triggering DAG
    - Via Airflow UI: Trigger DAG → Enter JSON configuration
    - Via CLI: airflow dags trigger fx_central_dag --conf '{...}'
    - Via API: POST /api/v1/dags/fx_central_dag/dagRuns with conf payload
2. **context['params']** - Fallback if conf empty
    - Defined in DAG instantiation: params={"default_config": "..."}
    - Less common for dynamic workflows

Schema Validation Benefits:

- **Early Failure:** Fails immediately if config invalid (before API calls)
- **Detailed Errors:** Provides JSON path to invalid field (e.g., "Task2 → method")
- **Type Safety:** Ensures downstream code receives expected types

Error Message Example:

Configuration validation failed: 'url' is a required property (at: Task1)

Input Preparation

def prepare_inputs(**context) -> Dict[str, Any]:
try:
validated_payload = validate_request_payload(**context)
correlation_id = (validated_payload.get('correlation_id') or '').strip() or 'UNKNOWN'
global_defaults = _extract_global_defaults(validated_payload)

```
    normalized = {}
    for n in range(1, 4):
        key = f'Task{n}'
        normalized[key] = _normalize_task_config(
            validated_payload[key],
            key,
            global_defaults,
            correlation_id,
        )

    logger.info("Preparing inputs for REST API calls and Kafka")
    logger.debug("Task1 config: %s", json.dumps(normalized['Task1'], indent=2
    logger.debug("Task2 config: %s", json.dumps(normalized['Task2'], indent=2
    logger.debug("Task3 config: %s", json.dumps(normalized['Task3'], indent=2
    return normalized
except AirflowException:
    raise
except Exception as e:
```

```
    logger.error("Unexpected error in prepare_inputs: %s", str(e), exc_info=Tru
    raise AirflowException(f"Failed to prepare inputs: {str(e)}") from e
```

Preparation Workflow:

1. **Validate** - Call validate_request_payload to load and validate config
2. **Extract Correlation ID** - Get tracking ID, default to 'UNKNOWN' if missing
3. **Extract Globals** - Pull global headers/timeout/verify_ssl
4. **Normalize Tasks** - Process Task1-3, applying global defaults and merging headers
5. **Return** - Return value is automatically pushed to XCom by Airflow

XCom Push: The returned `normalized` dictionary is stored in Airflow's metadata database and can be pulled by downstream tasks using `ti.xcom_pull(task_ids='prepare_inputs')`.

Normalized Output Structure:

```
{
'Task1': {
'url': '...',
'method': 'GET',
'headers': {...},
'json': None,
'timeout': 30.0,
'verify_ssl': True
},
'Task2': {...},
'Task3': {...}
}
```

Result Finalization

```
def finalize_results(**context) -> Dict[str, Any]:
try:
ti = context.get("ti")
if not ti:
raise AirflowException("Missing 'ti' (TaskInstance) in context")
```

```
    task_ids = [
        "task1_rest_api_call",
        "task2_rest_api_call",
        "task3_rest_api_call",
    ]
    results: Dict[str, Any] = {}
    failed_tasks: List[str] = []

    for task_id in task_ids:
        result = ti.xcom_pull(task_ids=task_id)
        results[f"{task_id.replace('_rest_api_call', '')}_result"] = result
        if result is None:
            failed_tasks.append(task_id)

    logger.info("Finalizing results from REST API calls")
    for key, value in results.items:
        logger.debug("%s: %s", key, json.dumps(value, indent=2) if value is not N

    status = "completed" if not failed_tasks else "completed_with_failures"

    finalized_results: Dict[str, Any] = {
        "task1_result": results.get("task1_result"),
        "task2_result": results.get("task2_result"),
        "task3_result": results.get("task3_result"),
        "failed_tasks": failed_tasks,
        "timestamp": datetime.now().isoformat(),
        "status": status,
    }
    logger.debug("Finalized results: %s", json.dumps(finalized_results, indent=
    return finalized_results
except AirflowException:
    raise
except Exception as e:
    logger.error("Unexpected error in finalize_results: %s", str(e), exc_info=Tru
    raise AirflowException(f"Failed to finalize results: {str(e)}") from e
```

Result Aggregation Logic:

1. **Get TaskInstance** - Extract ti from context to access XCom
2. **Pull Each Result** - Retrieve XCom from each API task
3. **Identify Failures** - If result is None, task failed (no XCom pushed)
4. **Determine Status** - "completed" if all succeeded, "completed_with_failures" otherwise
5. **Return Summary** - Aggregated results + failed task list + timestamp

Critical Design Choice: This task uses `TriggerRule.ALL_DONE` (see task definition section), meaning it runs regardless of upstream failures. This ensures we always have a summary, even if all API calls failed.

Output Example:

```
{
"task1_result": {
"status_code": 200,
"headers": {"content-type": "application/json"},
"response": {"users": [...]}
},
"task2_result": None, # Task2 failed
"task3_result": {
"status_code": 200,
"headers": {...},
"response": {"success": true}
},
"failed_tasks": ["task2_rest_api_call"],
"timestamp": "2026-03-02T09:45:23.123456",
"status": "completed_with_failures"
}
```

Section 4: Kafka Integration Functions

Kafka Envelope Builder

```
def _build_kafka_envelope(
event_type: str,
event_status: str,
outcome: Optional[str] = None,
```

summary: Optional[Dict[str, Any]] = None,
) -> str:
try:
context = get_current_context()
if not isinstance(context, dict):
raise ValueError("Context must be a dictionary")
dag_run: Optional[DagRun] = context.get("dag_run")
conf = (dag_run.conf or {}) if dag_run else {}
if not isinstance(conf, dict):
conf = {}

```
    payload: Dict[str, Any] = {
        "eventType": event_type,
        "run_control_id": conf.get("run_control_id") or "",
        "correlation_id": conf.get("correlation_id", "UNKNOWN"),
        "event_source": "AIRFLOW",
        "status": event_status,
        "trigger_payload": conf.get("trigger_payload") or "{}",
        "dagId": context["dag"].dag_id,
        "dagRunId": context["run_id"],
        "timestamp": _now_iso(),
    }
    if outcome is not None:
        payload["outcome"] = outcome
    if summary is not None:
        logger.info(f"Final summary of tasks : {summary}")
        payload["summary"] = summary

    return json.dumps(payload, default=str)
  except Exception as e:
    logger.error("Error building Kafka envelope: %s", str(e), exc_info=True)
    raise AirflowException(f"Failed to build Kafka message: {str(e)}") from e
```

Envelope Structure: Standardized CloudEvents-like format for workflow telemetry:

{
"eventType": "run.succeeded.v1",

"run_control_id": "custom-tracking-id",
"correlation_id": "req-12345",
"event_source": "AIRFLOW",
"status": "SUCCEEDED",
"trigger_payload": "{}",
"dagId": "fx_central_dag",
"dagRunId": "manual__2026-03-02T09:30:00+00:00",
"timestamp": "2026-03-02T09:45:23Z",
"outcome": "success",
"summary": {
"task1_result": {...},
"task2_result": {...},
"task3_result": {...},
"failed_tasks": [],
"status": "completed"
}
}

Key Fields:

- `eventType` - Versioned event name for consumer routing
- `correlation_id` - Links events across distributed systems
- `run_control_id` - Optional external tracking ID (for integration with upstream systems)
- `summary` - Only included in end event, contains task results

Kafka Start Producer

```
def kafka_producer_start(**kwargs: Any):
try:
message_json = _build_kafka_envelope(
event_type=EVENT_TYPE_RUN_STARTED,
event_status=EVENT_STATUS_RUNNING,
)
return [(None, message_json.encode("utf-8"))]
except Exception as e:
logger.error("Error in kafka_producer_start: %s", str(e),
exc_info=True)
raise AirflowException(f"Failed to build Kafka start message:
{str(e)}") from e
```

Producer Function Contract: Returns list of tuples `[(key, value)]`:

- `key=None` - No partition key (Kafka distributes round-robin)
- `value` - UTF-8 encoded JSON bytes

When It Executes: First task in DAG, runs immediately after trigger (before validation).

Kafka End Producer

def kafka_producer_end(**kwargs: Any):
try:
context = get_current_context()
ti = context.get("ti") if isinstance(context, dict) else None
if not ti:
raise AirflowException("Missing 'ti' (TaskInstance) in context")

```
final_summary = ti.xcom_pull(task_ids=TASK_ID_FINALIZE_RESULTS) or {}
if not isinstance(final_summary, dict):
    final_summary = {}

failed_tasks = final_summary.get("failed_tasks", [])
if not isinstance(failed_tasks, list):
    failed_tasks = []

if failed_tasks:
    event_type = EVENT_TYPE_RUN_FAILED
    event_status = EVENT_STATUS_FAILED
    outcome = OUTCOME_FAILED
else:
    event_type = EVENT_TYPE_RUN_SUCCEEDED
    event_status = EVENT_STATUS_SUCCEEDED
    outcome = OUTCOME_SUCCESS

message_json = _build_kafka_envelope(
    event_type=event_type,
    event_status=event_status,
    outcome=outcome,
    summary=final_summary,
```

```
    )
    return [(None, message_json.encode("utf-8"))]
except AirflowException:
    raise
except Exception as e:
    logger.error("Error in kafka_producer_end: %s", str(e), exc_info=True)
    raise AirflowException(f"Failed to build Kafka end message: {str(e)}") from
```

Dynamic Event Selection:

- Pulls `finalize_results` XCom
- Checks `failed_tasks` list
- If non-empty → Publishes `run.failed.v1` with outcome="failed"
- If empty → Publishes `run.succeeded.v1` with outcome="success"

Telemetry Pattern: Consumers can subscribe to specific event types:

- Alerting system subscribes to `run.failed.v1` for notifications
- Metrics system subscribes to all events for dashboard
- Audit system subscribes to all events for compliance logging

---

REST API Call Mechanism

Shared Execution Function

```
def _execute_api_call(context, task_key: str, task_label: str) -> Dict[str, Any]:
import requests
ti = context.get('ti')
if not ti:
raise AirflowException("Missing 'ti' (TaskInstance) in context")
payload = ti.xcom_pull(task_ids='prepare_inputs')
if not payload:
raise AirflowException("No payload from prepare_inputs; cannot run REST API call")
if task_key not in payload:
raise AirflowException(f"Payload missing {task_key}; cannot run REST API call")
```

```
config = payload[task_key]
url = config['url']
method = config['method']
headers = config.get('headers') or {}
json_payload = config.get('json')
verify_ssl = config.get('verify_ssl', DEFAULT_VERIFY_SSL)
timeout = config.get('timeout', DEFAULT_TIMEOUT_SECONDS)

logger.info("%s: Making %s request to %s", task_label, method, url)
try:
    response = requests.request(
        method=method,
        url=url,
        headers=headers,
        json=json_payload,
        verify=verify_ssl,
        timeout=timeout
    )
    response.raise_for_status()
    result = {
        'status_code': response.status_code,
        'headers': dict(response.headers),
        'response': response.json() if response.content else None
    }
    logger.info("%s completed successfully: status_code=%s", task_label, result
    return result
except requests.exceptions.HTTPError as e:
    logger.error("%s HTTP error: %s", task_label, str(e))
    raise AirflowException(f"{task_label} HTTP request failed: {str(e)}") from e
except requests.exceptions.ConnectionError as e:
    logger.error("%s connection error: %s", task_label, str(e))
    raise AirflowException(f"{task_label} HTTP request failed: {str(e)}") from e
except requests.exceptions.RequestException as e:
    logger.error("%s request failed: %s", task_label, str(e))
    raise AirflowException(f"{task_label} HTTP request failed: {str(e)}") from e
except Exception as e:
```

```
    logger.error("Unexpected error in %s: %s", task_label, str(e), exc_info=True
    raise AirflowException(f"{task_label} failed: {str(e)}") from e
```

Step-by-Step Execution Flow

1. **Import Requests Library**
    - Imported inside function (lazy import)
    - Python requests library handles HTTP communication[3]
2. **Get TaskInstance from Context**
    - context['ti'] provides access to XCom methods
    - Fails immediately if ti missing (defensive programming)
3. **Pull Normalized Config from XCom**
    - ti.xcom_pull(task_ids='prepare_inputs') retrieves dictionary
    - Returns None if prepare_inputs task failed
    - Raises error if payload missing or doesn't contain task key
4. **Extract Configuration Parameters**
    - config = payload[task_key] gets Task1/Task2/Task3 config
    - All parameters already validated and normalized
    - json_payload can be None (for GET requests)
5. **Execute HTTP Request**
    - requests.request() is generic method accepting any HTTP verb[3]
    - method='GET'/'POST'/etc. determines request type
    - headers dict sent as HTTP headers
    - json parameter automatically serializes to JSON and sets Content-Type
    - verify controls SSL certificate validation
    - timeout in seconds (raises Timeout exception if exceeded)
6. **Check HTTP Status**
    - response.raise_for_status() raises HTTPError for 4xx/5xx codes
    - 2xx codes proceed without exception
7. **Parse Response**
    - response.json() parses JSON body (if Content-Type: application/json)
    - Returns None if response.content empty (e.g., 204 No Content)
    - dict(response.headers) converts CaseInsensitiveDict to regular dict

8. **Return Result Dictionary**
    - Return value automatically pushed to XCom
    - Available to finalize_results task

Error Handling Hierarchy

Exception Type	Cause	Example
HTTPError	4xx/5xx status	404 Not Found, 500 Server Error
ConnectionError	Network failure	DNS resolution failed, refused
Timeout	Request too slow	Server unresponsive
RequestException	Generic failure	Redirect loop, invalid URL

Table 2: Requests library exception hierarchy

All exceptions are caught and re-raised as `AirflowException` to mark task as failed.

Task Wrapper Functions

```
def execute_task1_api_call(**context) -> Dict[str, Any]:
return _execute_api_call(context, 'Task1', 'Task1')

def execute_task2_api_call(**context) -> Dict[str, Any]:
return _execute_api_call(context, 'Task2', 'Task2')

def execute_task3_api_call(**context) -> Dict[str, Any]:
return _execute_api_call(context, 'Task3', 'Task3')
```

Why Separate Functions?

- Each task needs unique callable for PythonOperator
- `task_key` parameter selects correct config from normalized payload
- `task_label` parameter used for logging (identifies which task in logs)

XCom Data Passing

What is XCom?

XCom (Cross-Communication) is Airflow's mechanism for passing data between tasks[1]:

Storage:

- Data stored in Airflow metadata database (same DB as task state)
- Persists beyond task execution
- Scoped to DAG run (different runs have isolated XCom)

Size Limitations:

- Recommended maximum: 48KB (database BLOB field)
- Large data should use external storage (S3, GCS, database) with XCom storing reference

XCom Push Operations in This DAG

Task	Push Method	Data Pushed
prepare_inputs	return value	Normalized config dict
task1_rest_api_call	return value	API response dict
task2_rest_api_call	return value	API response dict
task3_rest_api_call	return value	API response dict
finalize_results	return value	Aggregated summary dict

Table 3: XCom push operations (automatic)

Automatic Push: When a Python function returns a value, Airflow automatically calls `ti.xcom_push(key='return_value', value=result)`.

XCom Pull Operations

Syntax:
ti.xcom_pull(task_ids='task_name') # Pulls 'return_value' key
ti.xcom_pull(task_ids='task_name', key='custom_key') # Pulls specific key

ti.xcom_pull(task_ids=['task1', 'task2']) # Pulls from multiple tasks (returns list)

Pull Operations in DAG:

1. **API Execution Tasks Pull from prepare_inputs**
   - payload = ti.xcom_pull(task_ids='prepare_inputs')
   - Gets normalized config dictionary
   - Each task extracts its own Task1/Task2/Task3 section
2. **finalize_results Pulls from All API Tasks**
   - result = ti.xcom_pull(task_ids='task1_rest_api_call')
   - Loops through all 3 API task IDs
   - Collects results into aggregated dictionary
3. **kafka_producer_end Pulls from finalize_results**
   - final_summary = ti.xcom_pull(task_ids='finalize_results')
   - Gets aggregated results to include in Kafka message

XCom Viewing in UI

You can inspect XCom values in Airflow UI:

Admin → XComs

- Filter by DAG ID and Run ID
- See all XCom keys and values for each task
- Useful for debugging data passing issues

Kafka Integration

Producer Operator Configuration

```
ProduceToTopicOperator(
task_id='publish_kafka_start',
topic='genesis.hub.run.events.v1',
kafka_config_id='genesis_kafka_conn',
producer_function=kafka_producer_start,
execution_timeout=KAFKA_EXECUTION_TIMEOUT,
poll_timeout=KAFKA_POLL_TIMEOUT,
)
```

Configuration Parameters:

- topic - Kafka topic name (versioned for schema evolution)
- kafka_config_id - Airflow connection ID (configured in UI: Admin → Connections)
- producer_function - Python callable returning [(key, value)] tuples
- execution_timeout - Max time for operator to complete (120 seconds)
- poll_timeout - Max wait for broker acknowledgment (60 seconds)

Kafka Connection Setup

In Airflow UI: Admin → Connections → Add Connection

- **Connection ID:** genesis_kafka_conn
- **Connection Type:** kafka
- **Extra:** JSON configuration:
  ```
  {
  "bootstrap.servers": "kafka-broker:9092",
  "security.protocol": "SASL_SSL",
  "sasl.mechanism": "PLAIN",
  "sasl.username": "user",
  "sasl.password": "pass"
  }
  ```

Event Flow

1. **Workflow Triggered**
   - User/System triggers DAG
   - publish_kafka_start executes immediately
   - Publishes run.started.v1 with status=RUNNING
2. **Workflow Executes**
   - REST API tasks run
   - Results collected
   - No Kafka messages during execution
3. **Workflow Completes**
   - finalize_results aggregates outcomes
   - publish_final_status checks for failures
   - Publishes run.succeeded.v1 or run.failed.v1
   - Includes full task summary in message

Consumer Use Cases

Alerting System:

Pseudo-code for Kafka consumer

```
for message in consumer.subscribe('genesis.hub.run.events.v1'):
event = json.loads(message.value)
if event['eventType'] == 'run.failed.v1':
send_alert(
title=f"Workflow Failed: {event['dagId']}",
correlation_id=event['correlation_id'],
failed_tasks=event['summary']['failed_tasks']
)
```

Metrics Dashboard:

Tracking workflow success rate

```
for message in consumer:
event = json.loads(message.value)
if event['eventType'] == 'run.succeeded.v1':
metrics.increment('workflow.success')
elif event['eventType'] == 'run.failed.v1':
metrics.increment('workflow.failure')
```

Error Handling Strategy

Trigger Rules

Default Rule: `TriggerRule.ALL_SUCCESS`

- Task runs only if ALL upstream tasks succeeded
- Applied to prepare_inputs and API tasks

Exception Rule: `TriggerRule.ALL_DONE`

- Task runs when ALL upstream tasks completed (success or failure)

- Applied to finalize_results and publish_final_status
- Ensures telemetry events always published

Failure Scenarios

Failure Point	Impact	Recovery
Invalid config	prepare_inputs fails	Kafka start published, end skipped
Task1 API fails	Task2/3 skipped	finalize_results runs, marks Task1 failed
Task2 API fails	Task3 skipped	finalize_results runs, marks Task2 failed
Task3 API fails	finalize_results runs	Marks Task3 failed
finalize_results fails	publish_final_status runs	No summary in Kafka message
Kafka publish fails	Workflow fails	Retry mechanism attempts 1 retry

Table 4: Failure scenarios and handling

Retry Behavior

Default Retry Configuration:
'retries': 1,
'retry_delay': timedelta(minutes=5),

Retry Logic:

1. Task fails with exception
2. Airflow waits 5 minutes
3. Automatically retries task once
4. If second attempt fails, marks task as failed permanently

Idempotency Consideration: API calls should be idempotent (safe to retry) or use unique request IDs to prevent duplicate processing.

Example Execution Scenarios

Scenario 1: All Tasks Succeed

Input Configuration:


```
{
"correlation_id": "req-success-001",
"global": {
"headers": {"Authorization": "Bearer token123"},
"timeout": 30
},
"Task1": {
"url": "https://api.example.com/users/1",
"method": "GET"
},
"Task2": {
"url": "https://api.example.com/orders",
"method": "POST",
"json": {"user_id": 1, "product": "widget"}
},
"Task3": {
"url": "https://api.example.com/notifications",
"method": "POST",
"json": {"message": "Order created"}
}
}
```


Execution Flow:

1. **publish_kafka_start**
   - Publishes: {"eventType": "run.started.v1", "status": "RUNNING", ...}
2. **prepare_inputs**
   - Validates config (passes)
   - Normalizes all 3 tasks
   - Pushes to XCom: {Task1: {url: "...", method: "GET", ...}, ...}
3. **task1_rest_api_call**
   - Pulls config from XCom
   - Executes GET https://api.example.com/users/1
   - Returns: {status_code: 200, response: {id: 1, name: "John"}}

4. **task2_rest_api_call**
    - Executes POST https://api.example.com/orders
    - Returns: {status_code: 201, response: {order_id: 456}}
5. **task3_rest_api_call**
    - Executes POST https://api.example.com/notifications
    - Returns: {status_code: 200, response: {success: true}}
6. **finalize_results**
    - Pulls all 3 results
    - failed_tasks = [] (empty)
    - status = "completed"
    - Returns aggregated summary
7. **publish_final_status**
    - Checks failed_tasks (empty)
    - Publishes: {"eventType": "run.succeeded.v1", "outcome": "success", "summary": {...}}

Final Kafka Message:

{
"eventType": "run.succeeded.v1",
"correlation_id": "req-success-001",
"status": "SUCCEEDED",
"outcome": "success",
"dagId": "fx_central_dag",
"dagRunId": "manual__2026-03-02T09:30:00+00:00",
"timestamp": "2026-03-02T09:45:23Z",
"summary": {
"task1_result": {"status_code": 200, "response": {"id": 1, "name": "John"}},
"task2_result": {"status_code": 201, "response": {"order_id": 456}},
"task3_result": {"status_code": 200, "response": {"success": true}},
"failed_tasks": [],
"status": "completed",
"timestamp": "2026-03-02T09:45:23.123456"
}
}

Scenario 2: Task2 Fails (500 Error)

Execution Flow:

1. publish_kafka_start, prepare_inputs, task1_rest_api_call execute successfully
2. **task2_rest_api_call**
   - Executes POST request
   - Server returns 500 Internal Server Error
   - response.raise_for_status() raises HTTPError
   - Airflow marks task as FAILED
   - XCom: No value pushed (return never reached)
3. **task3_rest_api_call**
   - SKIPPED (upstream failure + TriggerRule.ALL_SUCCESS)
4. **finalize_results** (TriggerRule.ALL_DONE)
   - Runs despite upstream failures
   - ti.xcom_pull(task_ids='task2_rest_api_call') returns None
   - failed_tasks = ["task2_rest_api_call", "task3_rest_api_call"]
   - status = "completed_with_failures"
5. **publish_final_status** (TriggerRule.ALL_DONE)
   - Checks failed_tasks (not empty)
   - Publishes: {"eventType": "run.failed.v1", "outcome": "failed"}

Final Kafka Message:


```
{
"eventType": "run.failed.v1",
"correlation_id": "req-failure-001",
"status": "FAILED",
"outcome": "failed",
"summary": {
"task1_result": {"status_code": 200, "response": {...}},
"task2_result": null,
"task3_result": null,
"failed_tasks": ["task2_rest_api_call", "task3_rest_api_call"],
"status": "completed_with_failures"
}
}
```

Scenario 3: Invalid Configuration

Input Configuration:

```
{
"correlation_id": "req-invalid-001",
"Task1": {
"url": "https://api.example.com/users",
"method": "GET"
},
"Task2": {
"url": "INVALID_URL",
"method": "POST"
}
}
```

Execution Flow:

1. **publish_kafka_start**
   - Succeeds (publishes before validation)
2. **prepare_inputs**
   - validate_request_payload detects missing Task3
   - jsonschema.validate raises ValidationError
   - Re-raised as AirflowException
   - Task marked FAILED
3. **All downstream tasks**
   - SKIPPED (prepare_inputs failed)
   - finalize_results skipped despite TriggerRule.ALL_DONE (because prepare_inputs is ancestor)
4. **publish_final_status**
   - NOT EXECUTED (finalize_results didn't run)

Result: Only Kafka start event published, workflow marked as failed in Airflow UI.

DAG Definition and Task Operators

DAG Instantiation

```
dag = DAG(
'fx_central_dag',
default_args=default_args,
description='DAG with 3 REST API calls and Kafka integration',
schedule=None,
start_date=datetime(2024, 1, 1),
catchup=False,
tags=['rest_api', 'kafka', 'fx_central_dag'],
)
```

Parameters:

- `schedule=None` - Manual trigger only (no automatic scheduling)
- `start_date` - Historical date (irrelevant since catchup=False)
- `catchup=False` - Don't backfill historical runs
- `tags` - Searchable labels in UI

Task Definitions

```
publish_kafka_start_task = ProduceToTopicOperator(
task_id='publish_kafka_start',
topic='genesis.hub.run.events.v1',
kafka_config_id='genesis_kafka_conn',
producer_function=kafka_producer_start,
execution_timeout=KAFKA_EXECUTION_TIMEOUT,
poll_timeout=KAFKA_POLL_TIMEOUT,
dag=dag,
)
```

Kafka Start Task: Publishes workflow initiation event before any processing.

```
prepare_inputs_task = PythonOperator(
task_id='prepare_inputs',
python_callable=prepare_inputs,
trigger_rule=TriggerRule.ALL_DONE,
dag=dag,
)
```

Input Preparation Task:

- Uses `TriggerRule.ALL_DONE` to run even if Kafka start fails
- Validates and normalizes configuration

```
task1_rest_api_call = PythonOperator(
task_id='task1_rest_api_call',
python_callable=execute_task1_api_call,
trigger_rule=TriggerRule.ALL_DONE,
dag=dag,
)

task2_rest_api_call = PythonOperator(
task_id='task2_rest_api_call',
python_callable=execute_task2_api_call,
dag=dag,
)

task3_rest_api_call = PythonOperator(
task_id='task3_rest_api_call',
python_callable=execute_task3_api_call,
dag=dag,
)
```

API Tasks: Execute REST calls sequentially using normalized configs from XCom.

```
finalize_results_task = PythonOperator(
task_id='finalize_results',
python_callable=finalize_results,
trigger_rule=TriggerRule.ALL_DONE,
dag=dag,
)
```

Result Finalization: Aggregates outcomes, runs regardless of API task success.

```
publish_final_status_task = ProduceToTopicOperator(
task_id='publish_final_status',
topic='genesis.hub.run.events.v1',
kafka_config_id='genesis_kafka_conn',
producer_function=kafka_producer_end,
```

```
trigger_rule=TriggerRule.ALL_DONE,
execution_timeout=KAFKA_EXECUTION_TIMEOUT,
poll_timeout=KAFKA_POLL_TIMEOUT,
dag=dag,
)
```

Kafka End Task: Publishes completion event with success/failure status.

Task Dependencies

```
publish_kafka_start_task >> prepare_inputs_task >>
task1_rest_api_call >> task2_rest_api_call >> task3_rest_api_call >>
finalize_results_task >> publish_final_status_task
```

Dependency Chain: Linear pipeline using `>>` operator (bitshift syntax).

Alternative Syntax:

Equivalent using set_downstream

```
publish_kafka_start_task.set_downstream(prepare_inputs_task)
prepare_inputs_task.set_downstream(task1_rest_api_call)
```

... etc

Or set_upstream

```
task1_rest_api_call.set_upstream(prepare_inputs_task)
```

Key Takeaways

Design Patterns

1. **Configuration Normalization**
    - Global defaults + task-specific overrides
    - Header merging with correlation ID injection
    - Defensive validation (type checks, fallbacks)
2. **XCom-Based Data Flow**
    - prepare_inputs pushes once, all API tasks pull
    - Reduces duplication and ensures consistency
    - Enables result aggregation in finalize_results
3. **Resilient Telemetry**
    - TriggerRule.ALL_DONE ensures completion events
    - Dynamic event type based on outcome
    - Full task summary in Kafka messages
4. **Early Validation**
    - JSON schema validation before API execution
    - Fails fast with descriptive errors
    - Prevents wasted API calls with bad config

Extensibility

Adding More Tasks:

1. Add `Task4` to CONFIG_SCHEMA
2. Create `execute_task4_api_call` function
3. Add task to normalization loop in `prepare_inputs`
4. Create task4_rest_api_call operator
5. Add to dependency chain
6. Update `finalize_results` to include Task4

Adding Parallel Execution:

Current: Sequential

task1 >> task2 >> task3

Parallel: All run simultaneously

prepare_inputs_task >> [task1_rest_api_call, task2_rest_api_call, task3_rest_api_call] >> finalize_results_task

Adding Custom Headers Per Task:
Already supported - specify in Task1/Task2/Task3 config:

```
{
"Task1": {
"url": "...",
"method": "GET",
"headers": {
"X-Custom-Header": "value"
}
}
}
```

Production Considerations

1. **Secrets Management**
   - Don't hardcode tokens in configuration
   - Use Airflow Variables or Connections for credentials
   - Reference in DAG: Variable.get("api_token")
2. **Monitoring**
   - Subscribe to Kafka topic for real-time monitoring
   - Set up alerts for run.failed.v1 events
   - Track correlation_id across systems
3. **Idempotency**
   - API calls should be safe to retry
   - Use unique request IDs in headers
   - Implement server-side deduplication
4. **Timeout Tuning**
   - Set appropriate timeouts per task (slow APIs need longer)
   - Consider network latency and payload size
   - Balance between responsiveness and reliability

References

[1] Astronomer. (2019). Pass data between tasks. *Astronomer Documentation*. https://www.astronomer.io/docs/learn/airflow-passing-data-between-tasks

[2] Stack Overflow. (2020). Airflow: Trigger DAG via UI with Parameters/Config. https://stackoverflow.com/questions/60556380/airflow-trigger-dag-via-ui-with-parameters-config

[3] Real Python. (2025). Python's Requests Library (Guide). https://realpython.com/python-requests/